

Diseño de interfaces Web

UD4: Integración de contenido interactivo

Caso Práctico 1: Integración de contenido multimedia e interactivo de una web de educación online

Índice

1. [Introducción](#)
2. [Estructura general de la aplicación](#)
3. [Navegación por capítulos y control del tiempo del vídeo](#)
4. [Activación automática del cuestionario](#)
 - 4.1. [Activación del cuestionario al finalizar el vídeo](#)
 - 4.2. [Evaluación de respuestas y retroalimentación](#)
 - 4.3. [Bloqueo del cuestionario y reintento condicionado](#)
5. [Integración de audio y subtítulos dinámicos](#)
6. [Diseño visual y adaptación responsiva](#)
7. [Elección de formatos multimedia](#)
8. [Conclusión](#)
9. [Referencias](#)

1. Introducción

El presente trabajo consiste en el desarrollo de una aplicación web interactiva orientada a la integración de contenidos multimedia y a la aplicación de lógica mediante JavaScript.

El objetivo principal es demostrar el uso de los elementos multimedia de HTML5 (`<video>` y `<audio>`), la gestión de eventos, la manipulación del DOM y la adaptación del diseño a distintos tamaños de pantalla mediante CSS responsivo.

La aplicación incorpora un vídeo educativo, un sistema de navegación por capítulos, un cuestionario de evaluación que se activa al finalizar la reproducción y un recurso de audio con subtítulos sincronizados dinámicamente.

El proyecto se ha desarrollado utilizando únicamente tecnologías del lado del cliente: HTML5, CSS3 y JavaScript.

2. Estructura general de la aplicación

La aplicación se ha estructurado utilizando elementos semánticos de HTML5 con el objetivo de organizar correctamente el contenido y facilitar su comprensión.

El contenedor principal `<main>` agrupa las distintas funcionalidades del sistema. Dentro de él, se emplean elementos `<article>` para separar las secciones principales: vídeo educativo con navegación por capítulos y cuestionario dinámico, y recurso de audio con subtítulos sincronizados.

La visibilidad del cuestionario se controla mediante la clase `oculto`, que se añade o elimina dinámicamente desde JavaScript en función del estado de reproducción del vídeo. De este modo, la interfaz responde de forma controlada a los eventos generados por la interacción del usuario.

La lógica de comportamiento se encuentra en un archivo JavaScript independiente, mientras que los estilos visuales se gestionan desde un archivo CSS externo, favoreciendo la separación de responsabilidades.

```
<main class="page">
  <h1>Demo - Multimedia en Educación Online</h1>

  <article>
    <h2>Vídeo educativo</h2>

    <video id="videoCurso" controls preload="metadata" width="100%">
      <source src="media/video.mp4" type="video/mp4" />
      Tu navegador no soporta la etiqueta <code>video</code>.
    </video>
```

```

<p class="hint" id="estadoVideo" aria-live="polite">
  Reproduce el vídeo. Al finalizar, aparecerá un cuestionario.
</p>

[...]

<div id="quiz" class="oculto">
  [...]
</div>

[...]

<p>
  En este ejemplo se integra vídeo con HTML5, se añaden "capítulos" (saltos por
  tiempo) y se muestra un cuestionario al terminar la reproducción.
</p>
</article>

<article>
  <h2>Audio de conferencia</h2>

  <audio id="audioConferencia" controls preload="metadata">
    <source src="media/audio.mp3" type="audio/mpeg" />
    Tu navegador no soporta la etiqueta <code>audio</code>.
  </audio>

  [...]
</article>
</main>

```

3. Navegación por capítulos y control del tiempo del vídeo

Se ha implementado un sistema de navegación por capítulos que permite desplazarse a momentos concretos del vídeo sin necesidad de reproducirlo desde el inicio. Este mecanismo mejora la experiencia de usuario al facilitar el acceso directo a contenidos específicos, especialmente en entornos educativos donde puede ser necesario revisar un punto determinado. Este sistema mejora la usabilidad al permitir al usuario acceder directamente a los contenidos relevantes sin necesidad de reproducir el vídeo completo, facilitando una navegación más eficiente.

Cada botón de la tabla incorpora un atributo personalizado *data-time*, que almacena el segundo exacto al que debe saltar la reproducción. El uso de atributos *data-** permite asociar información adicional a elementos HTML sin alterar la estructura semántica del documento, manteniendo una separación clara entre contenido y comportamiento.

Mediante JavaScript, se captura el evento *click* sobre la tabla y se identifica el botón seleccionado utilizando el método *closest()*. A partir del valor almacenado en

dataset.time, el dato se convierte a número y se utiliza para modificar la propiedad *currentTime* del elemento *<video>*. Esta propiedad permite establecer directamente la posición de reproducción dentro del recurso multimedia.

La operación de cambio de tiempo y el intento de reproducción automática se encapsulan dentro de un bloque *try/catch*. Esta decisión busca aumentar la robustez del sistema, evitando posibles errores derivados de diferencias de implementación entre navegadores o restricciones de reproducción automática impuestas por algunos entornos.

Finalmente, se actualiza dinámicamente el texto de estado para informar al usuario del salto realizado, proporcionando retroalimentación inmediata sobre la acción ejecutada.

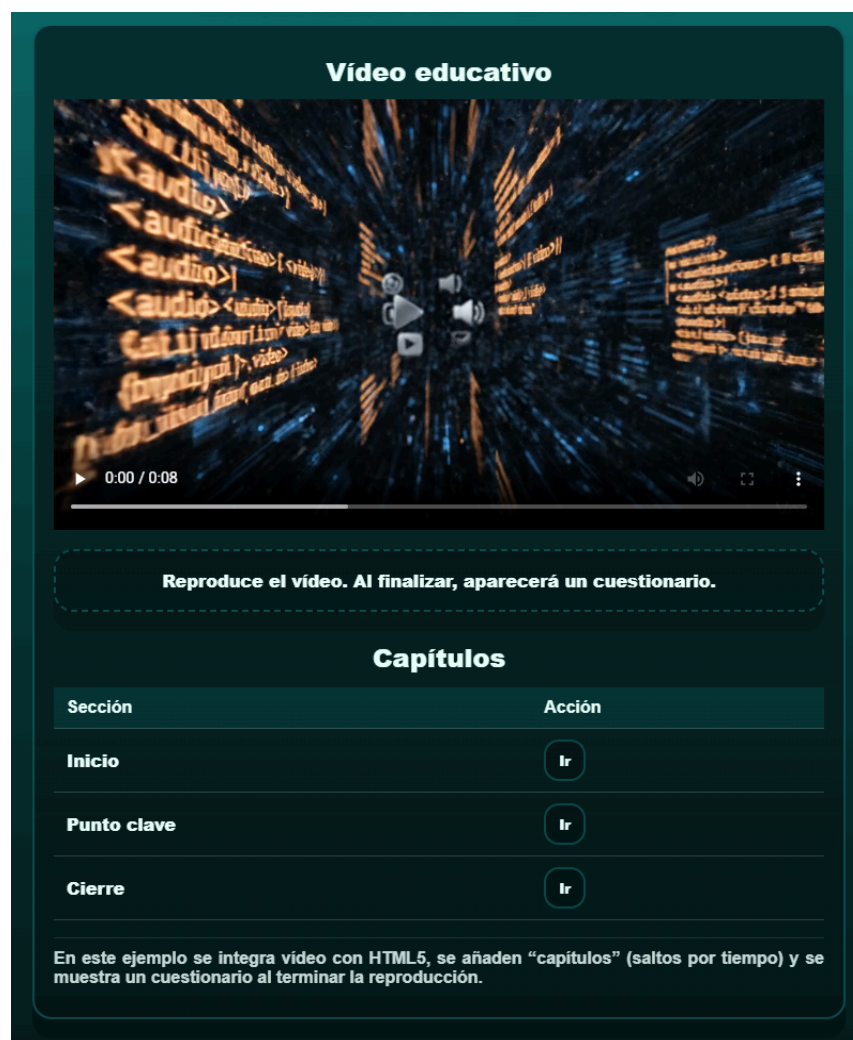


Figura 1. Interfaz inicial del vídeo educativo antes de la reproducción, mostrando la tabla de capítulos y el mensaje informativo.

El funcionamiento interno del sistema se basa en la captura del evento *click* sobre la tabla de capítulos. En lugar de añadir un listener individual a cada botón, se emplea

delegación de eventos, lo que permite optimizar el código y facilitar su mantenimiento.

A continuación se muestra el fragmento responsable de gestionar el salto temporal y la actualización del mensaje de estado:

```
if (tablaCapitulos && video) {
  tablaCapitulos.addEventListener("click", (e) => {
    const btn = e.target.closest("button[data-time]");
    if (!btn) return;

    const t = Number(btn.dataset.time);
    if (Number.isNaN(t)) return;

    try {
      video.currentTime = t;
      const p = video.play();
      if (p && typeof p.catch === "function") p.catch(() => {});
    } catch (_) {}

    if (estadoVideo) estadoVideo.textContent = `Saltaste a ${t}s (capítulo).`;
  });
}
```

Tras la ejecución del salto, el mensaje de estado se actualiza dinámicamente para informar al usuario del segundo exacto al que se ha desplazado la reproducción.

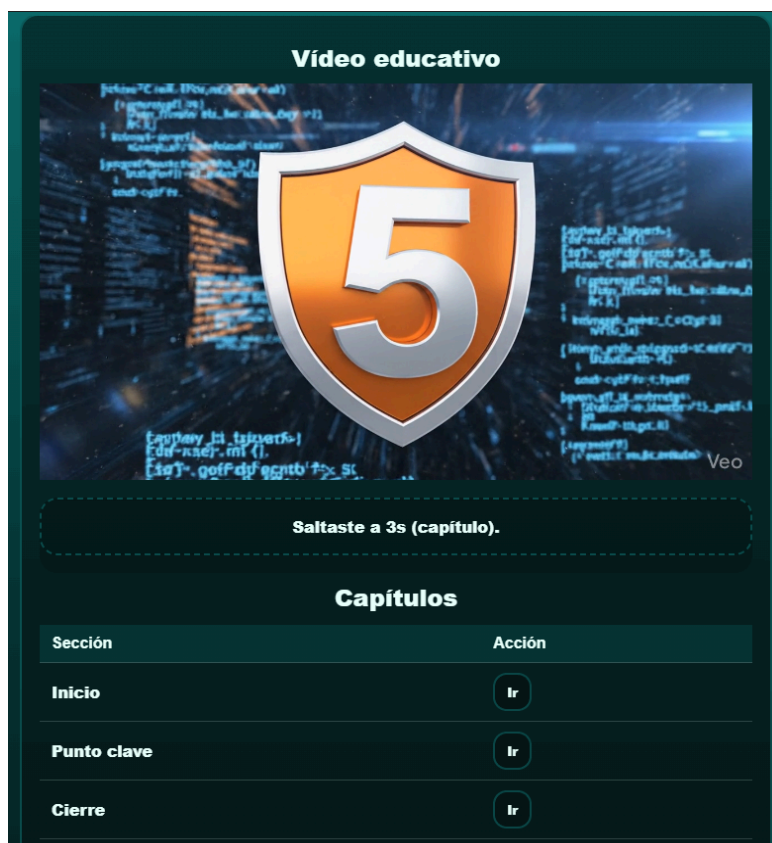


Figura 2. Estado del vídeo tras seleccionar un capítulo, con actualización dinámica del mensaje indicando el salto temporal.

4. Activación automática del cuestionario

El cuestionario se encuentra inicialmente oculto mediante la clase *oculto*, evitando que el usuario lo complete antes de finalizar la visualización del vídeo. Este funcionamiento se basa en el modelo de programación orientada a eventos propio de JavaScript, donde determinadas acciones del usuario o del sistema activan funciones específicas.

Cuando el vídeo termina, se detecta el evento *ended* y se ejecuta una función que muestra el cuestionario y prepara la interfaz para responder. Tras el envío del formulario, las respuestas se evalúan y se muestra retroalimentación (acierto/error por pregunta y resultado global). Finalmente, el sistema bloquea el cuestionario para impedir que se repita inmediatamente, obligando a volver a reproducir el vídeo para realizar un nuevo intento.

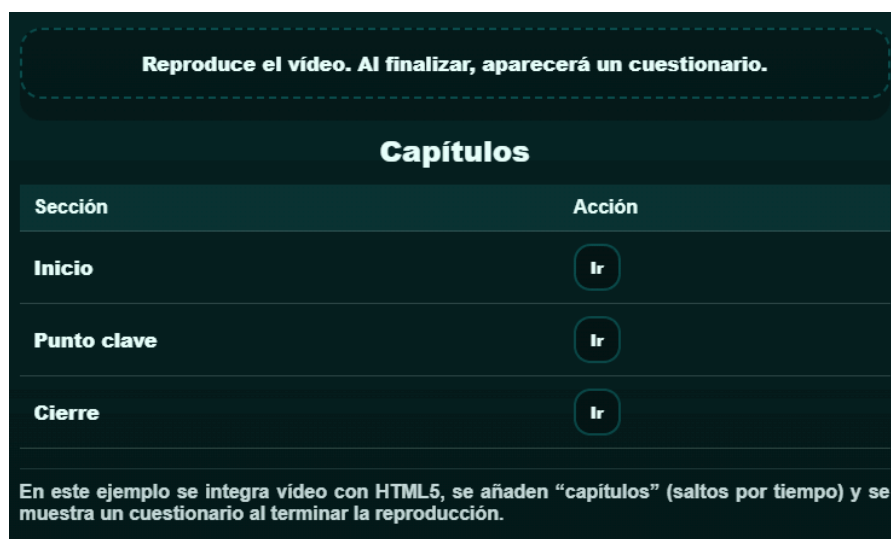


Figura 3. Estado previo a la finalización del vídeo, con el cuestionario aún oculto.

Cuestionario (al finalizar el vídeo)

- ¿Qué etiqueta HTML5 se usa para insertar audio?
- ¿Qué formato de vídeo ofrece mayor compatibilidad entre navegadores?
- ¿Qué evento de JavaScript se usa para detectar que el vídeo ha terminado?

Comprobar

En este ejemplo se integra vídeo con HTML5, se añaden “capítulos” (saltos por tiempo) y se muestra un cuestionario al terminar la reproducción.

Figura 4. Cuestionario visible tras la detección del evento *ended*, preparado para su cumplimentación.

4.1 Activación del cuestionario al finalizar el vídeo

La activación del cuestionario se basa en la detección del evento *ended* del elemento `<video>`. Este evento se dispara automáticamente cuando la reproducción alcanza el final.

En el siguiente fragmento se observa cómo se escucha dicho evento y se ejecuta la función *mostrarQuiz()* al producirse:

```
if (video) {  
  video.addEventListener("ended", () => {  
    mostrarQuiz();  
  });  
}
```

De esta manera, el cuestionario no depende de una acción manual del usuario, sino que se muestra únicamente cuando el contenido ha sido visualizado por completo.

La función *mostrarQuiz()* es la encargada de modificar dinámicamente la interfaz cuando el vídeo finaliza.

En el fragmento siguiente se aprecia cómo se elimina la clase `oculto`, haciendo visible el contenedor del cuestionario, y cómo se actualiza el mensaje de estado para informar al usuario:

```
function mostrarQuiz() {  
  [...]  
  quiz.classList.remove("oculto");  
  estadoVideo.textContent = "Vídeo finalizado. Completa el cuestionario para comprobar  
la comprensión."  
  [...]  
}
```

Adicionalmente, esta función puede incluir otras acciones complementarias, como el desplazamiento automático hacia el formulario o la preparación del sistema para un nuevo intento.

4.2 Evaluación de respuestas y retroalimentación

La evaluación del cuestionario se activa cuando el usuario envía el formulario. En ese momento, se evita la recarga de la página y se procesan las respuestas seleccionadas.

```
if (quizForm) {  
  quizForm.addEventListener("submit", (e) => {  
    e.preventDefault();  
  });  
}
```

```
if (quizLocked) return;

const data = new FormData(quizForm);
const keys = Object.keys(Respuestas);

let puntos = 0;

[...]
```

Cada respuesta se compara con un conjunto de soluciones almacenadas en un objeto de referencia. Por cada coincidencia, se incrementa el contador de aciertos y se genera una indicación visual para cada pregunta.

Cuestionario (al finalizar el vídeo)

1. ¿Qué etiqueta HTML5 se usa para insertar audio?

✓ Correcto

2. ¿Qué formato de vídeo ofrece mayor compatibilidad entre navegadores?

✓ Correcto

3. ¿Qué evento de JavaScript se usa para detectar que el vídeo ha terminado?

✗ Incorrecto

Reproduce el vídeo para reintentar

Resultado: 2 / 3. 🔄 Puedes reintentar reproduciendo el vídeo.

Figura 5. Cuestionario tras la evaluación con respuestas incorrectas, mostrando retroalimentación individual y resultado global.

Tras evaluar todas las preguntas, se muestra un resultado global con el número de respuestas correctas obtenidas.

Cuestionario (al finalizar el vídeo)

1. ¿Qué etiqueta HTML5 se usa para insertar audio?

<audio>
▼

✔ **Correcto**

2. ¿Qué formato de vídeo ofrece mayor compatibilidad entre navegadores?

MP4 (H.264)
▼

✔ **Correcto**

3. ¿Qué evento de JavaScript se usa para detectar que el vídeo ha terminado?

ended
▼

✔ **Correcto**

Reproduce el vídeo para reintentar

Resultado: 3 / 3. ✔ Excelente comprensión.

Figura 6. Cuestionario tras la evaluación con todas las respuestas correctas, indicando resultado completo y confirmación visual.

Finalmente, el sistema bloquea el formulario para impedir nuevos intentos sin volver a reproducir el vídeo.

```

if (quizFeedback) {
    quizFeedback.classList.remove("oculto");
    quizFeedback.textContent =
        `Resultado: ${puntos} / ${keys.length}. ` +
        (puntos === keys.length
            ? "✔ Excelente comprensión."
            : "🔄 Puedes reintentar reproduciendo el vídeo.");
}

lockQuizUI();

```

4.3 Bloqueo del cuestionario y reintento condicionado

Tras la evaluación, el sistema modifica el estado del formulario para impedir nuevos intentos inmediatos. Este cambio afecta tanto a la interfaz visible como al DOM.

El botón de envío pasa a estar deshabilitado mediante el atributo *disabled*, lo que impide su activación desde el navegador.

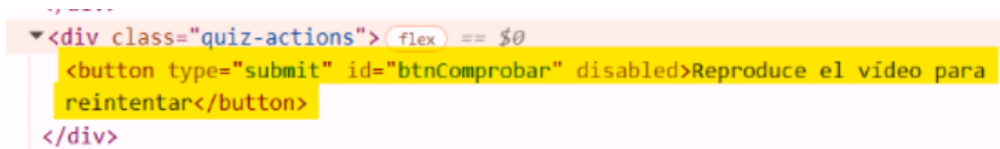


Figura 7. Botón de envío deshabilitado tras la evaluación, impidiendo nuevos intentos inmediatos.

Además, el formulario incorpora la clase *is-locked*, lo que permite aplicar estilos específicos y controlar su estado desde JavaScript.

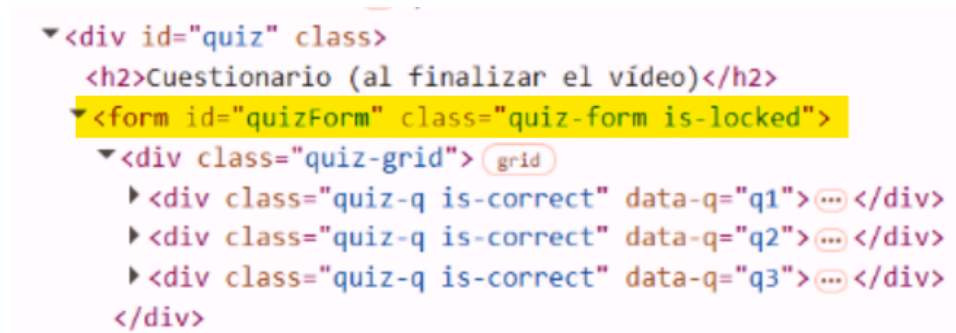


Figura 8. Aplicación de la clase *is-locked* en el formulario, reflejando el estado bloqueado en el DOM.

El desbloqueo se produce cuando el usuario vuelve a reproducir el vídeo. Para ello, se escucha el evento *play*, que restaura el estado interactivo del formulario.

```
if (video) {
  video.addEventListener("ended", () => {
    mostrarQuiz();
  });

  video.addEventListener("play", () => {
    if (quizLocked) unlockQuizUI();
  });
}
```

5. Integración de audio y subtítulos dinámicos

La aplicación incorpora un recurso de audio independiente del vídeo principal, utilizando la etiqueta HTML5 *<audio>*. Este elemento permite la reproducción nativa sin necesidad de complementos externos.

El audio se acompaña de un sistema de subtítulos dinámicos gestionado mediante JavaScript. A través del evento *timeupdate*, se detecta el momento exacto de

reproducción y se actualiza el texto mostrado en pantalla en función del tiempo transcurrido.

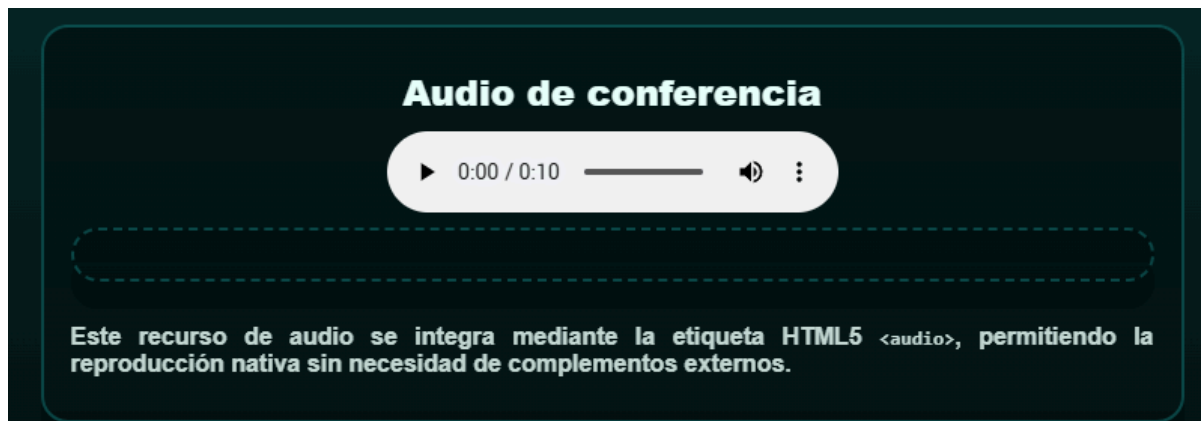


Figura 9. Recurso de audio integrado mediante la etiqueta <audio>, antes del inicio de la reproducción.

Cuando el audio comienza a reproducirse, el contenido textual cambia automáticamente según los intervalos definidos en el script.

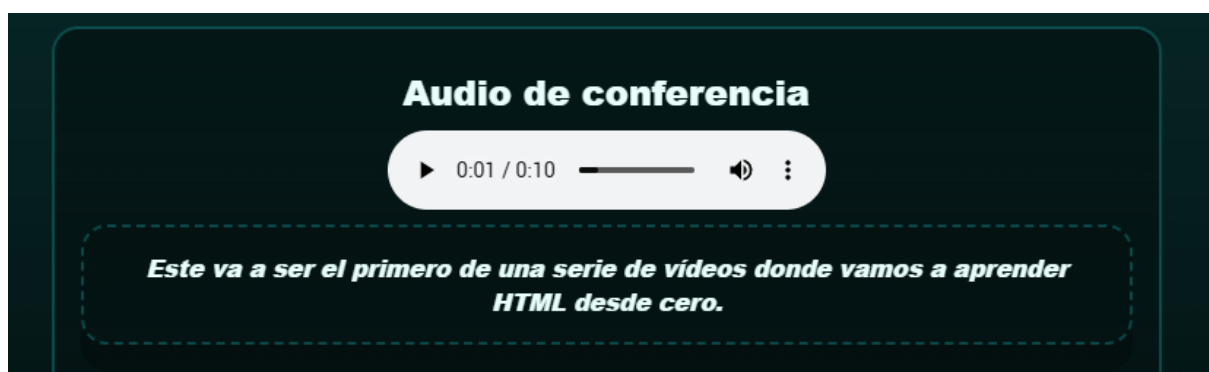


Figura 10. Actualización dinámica del subtítulo durante la reproducción del audio (primer intervalo temporal).

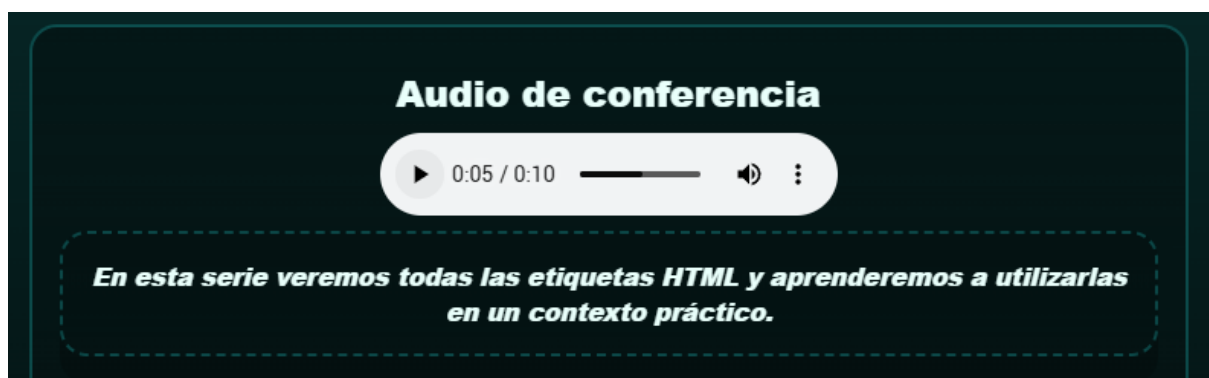


Figura 11. Cambio automático del subtítulo en un intervalo posterior de la reproducción.

La actualización del texto se realiza consultando la propiedad *currentTime* del elemento `<audio>`, comparándola con rangos temporales predefinidos y modificando dinámicamente el contenido del elemento destinado a mostrar los subtítulos.

```

audio.addEventListener("timeupdate", () => {
  const t = audio.currentTime;

  if (t < 5) {
    audioCaption.textContent = "Este va a ser el primero de una serie de vídeos donde vamos a aprender HTML desde cero.";
  } else if (t < 10) {
    audioCaption.textContent = "En esta serie veremos todas las etiquetas HTML y aprenderemos a utilizarlas en un contexto práctico.";
  } else {
    audioCaption.textContent = "";
  }
});

```

6. Diseño visual y adaptación responsiva

El diseño visual de la aplicación se ha desarrollado mediante CSS externo, separando completamente la lógica de comportamiento de la presentación. Esta separación facilita el mantenimiento del código y mejora la organización general del proyecto.

Se han aplicado estilos personalizados para estructurar el contenido en bloques diferenciados, utilizando propiedades como *flex*, *grid*, sombras, bordes redondeados y transiciones visuales que refuerzan la claridad de la interfaz.

La disposición de los elementos se adapta a distintos tamaños de pantalla mediante el uso de reglas *@media*. Estas permiten modificar tamaños de texto, distribución de formularios y anchura de botones cuando el ancho de la ventana se reduce.

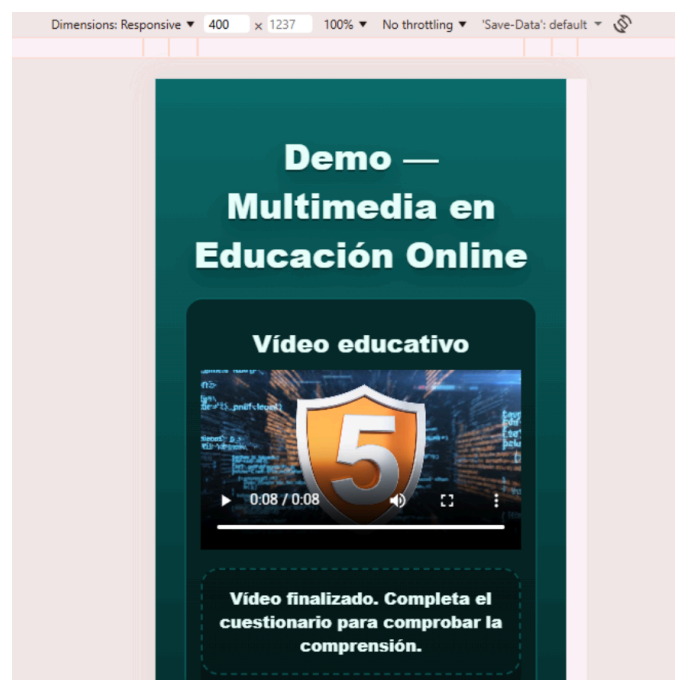


Figura 12. Visualización en dispositivo móvil (aprox. 400px), con disposición vertical optimizada y botones a ancho completo.



Figura 13. Adaptación del diseño en resolución intermedia (aprox. 820px), con reorganización del formulario y ajuste de tamaños.



Figura 14. Visualización de la interfaz en resolución de escritorio (aprox. 1080px), mostrando distribución completa de los elementos.

7. Elección de formatos multimedia

Para el recurso de vídeo se ha utilizado el formato MP4 con códec H.264, ya que ofrece una alta compatibilidad entre navegadores modernos y dispositivos móviles. Este formato permite una buena relación entre calidad y tamaño de archivo, lo que resulta adecuado para entornos educativos en línea.

En el caso del audio, se ha empleado el formato MP3 debido a su amplia aceptación y soporte nativo en navegadores actuales. Su compresión eficiente facilita la distribución del recurso sin comprometer de forma significativa la calidad sonora.

La selección de ambos formatos responde a criterios de interoperabilidad, rendimiento y accesibilidad, asegurando que el contenido pueda reproducirse sin requerir configuraciones adicionales por parte del usuario.

8. Conclusión

La aplicación desarrollada integra distintos recursos multimedia mediante tecnologías estándar de HTML5, combinando vídeo, audio e interacción dinámica en un mismo entorno.

A través del uso de eventos de JavaScript se ha conseguido controlar el flujo de interacción, mostrando y bloqueando el cuestionario según el estado de reproducción del vídeo. Asimismo, se ha implementado un sistema de retroalimentación inmediata que refuerza el aprendizaje mediante validación automática.

El diseño responsivo garantiza que la aplicación pueda utilizarse en diferentes dispositivos sin pérdida de funcionalidad, mientras que la elección de formatos compatibles asegura su correcta reproducción en navegadores actuales.

En conjunto, el proyecto demuestra la integración coordinada de estructura semántica, estilos externos y lógica de comportamiento, aplicando buenas prácticas de separación de responsabilidades y organización del código.

9. Referencias

- MDN Web Docs. Documentación oficial sobre HTML5, elementos multimedia y eventos del DOM.
<https://developer.mozilla.org/es/>
- W3Schools. Recursos de consulta para etiquetas HTML, eventos y diseño responsivo.
<https://www.w3schools.com/>

- Mimo App. Aplicación móvil utilizada para reforzar la práctica de conceptos de HTML, CSS y JavaScript.
- Codewars. Plataforma de práctica de programación empleada para consolidar lógica en JavaScript.
<https://www.codewars.com/>
- Herramientas de desarrollo del navegador (DevTools) utilizadas para la inspección del DOM y depuración del comportamiento dinámico.
- Apuntes de la unidad didáctica de Desarrollo Web en Entorno Cliente.
- Vídeo educativo consultado en YouTube para la extracción de un fragmento de audio utilizado con fines académicos.
<https://www.youtube.com/watch?v=IZeH53-EdjA>
- Google Gemini. Herramienta empleada para la generación del recurso audiovisual utilizado en el proyecto.